

Cyprus University of Technology

CSE 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Κλάσεις και αντικείμενα στην Java
Strings

Παναγιώτης Ηλία

ΚΛΑΣΕΙΣ ΚΑΙ ΑΝΤΙΚΕΙΜΕΝΑ

Κλάση

- Μια κλάση είναι:
  - Μία αφηρημένη περιγραφή αντικειμένων με κοινά χαρακτηριστικά και κοινή συμπεριφορά
  - Ένα καλούπι/πρότυπο που παράγει αντικείμενα
- Ένα αντικείμενο είναι:
  - Ενα στιγμιότυπο μίας κλάσης
- Η κλάση ορίζει τον τύπο του αντικειμένου
  - Τα χαρακτηριστικά του αντικειμένου
  - Τις ενέργειες που μπορεί να εκτελέσει

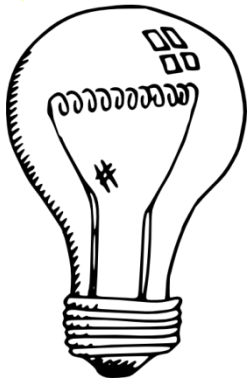
Πρακτικά στον κώδικα

- Μία κλάση **Κ** ορίζεται από
  - Κάποιες μεταβλητές τις οποίες ονομάζουμε πεδία
  - Κάποιες συναρτήσεις που τις ονομάζουμε μεθόδους
    - Οι μέθοδοι «βλέπουν» τα πεδία της κλάσης

} Μέλη της κλάσης

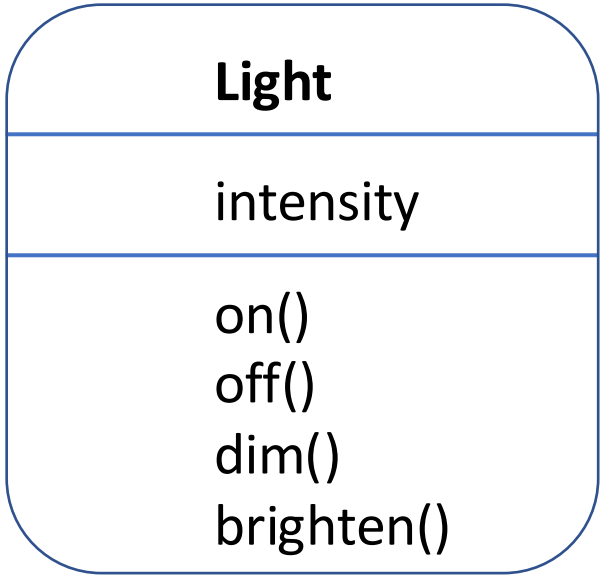
- Ένα αντικείμενο ορίζεται ως μια μεταβλητή τύπου Κ
  - Το αντικείμενο έχει συγκεκριμένες τιμές στα πεδία
  - Στο πρόγραμμα έχουμε (συνήθως) πρόσβαση μόνο τις μεθόδους
    - Μέσω των μεθόδων έχουμε πρόσβαση στα πεδία
  - Αν υπάρχουν κάποια πεδία στα οποία έχουμε πρόσβαση αυτά τα λέμε properties

Γενική μορφή της κλάσης



Θέλουμε να κάνουμε ένα πρόγραμμα που θα διαχειρίζεται τα φώτα σε διάφορα δωμάτια και επίσης θα υλοποιεί ένα dimmer

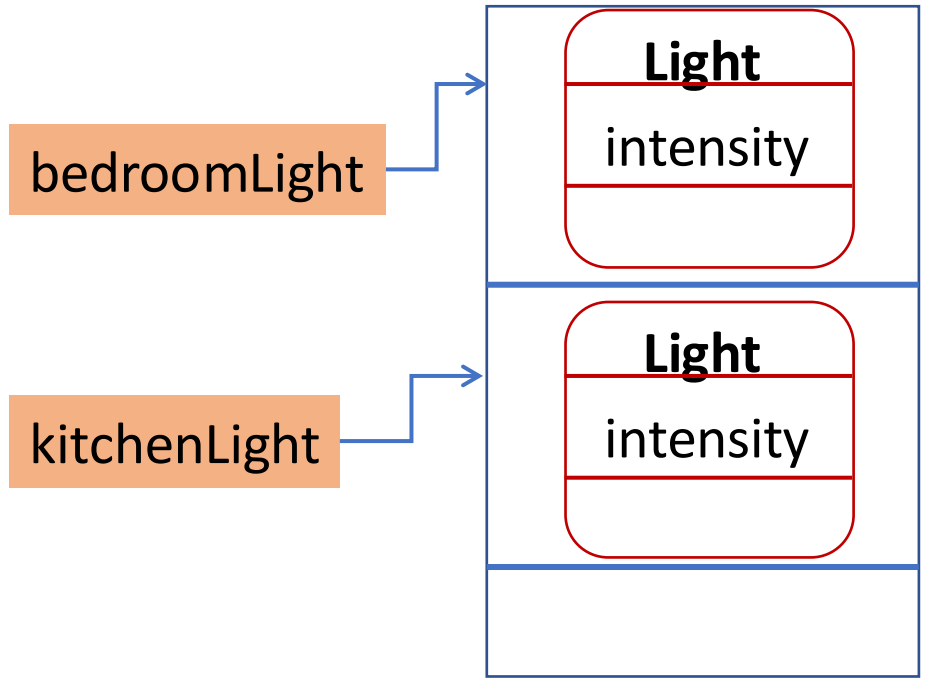




Δημιουργία Αντικειμένων

```
Light bedroomLight = new Light();
Light kitchenLight = new Light();
```

- Με την εντολή new δημιουργούμε ένα καινούριο αντικείμενο της κλάσης και του δίνουμε ένα όνομα
- Η new δεσμεύει χώρο μνήμης για το αντικείμενο
  - Μας επιστρέφει την διεύθυνση του χώρου που δεσμεύτηκε
- Η μεταβλητή που ορίζουμε «δείχνει» σε αυτό τον χώρο μνήμης



Κλήση Μεθόδων

- Ή πρόσβαση που έχουμε στα αντικείμενα είναι (κατά κύριο λόγο) μέσα από τις μεθόδους τους
- Η κλήση μιας μεθόδου
  - `<όνομα αντικειμένου>.<όνομα μεθόδου>`
- Π.χ.

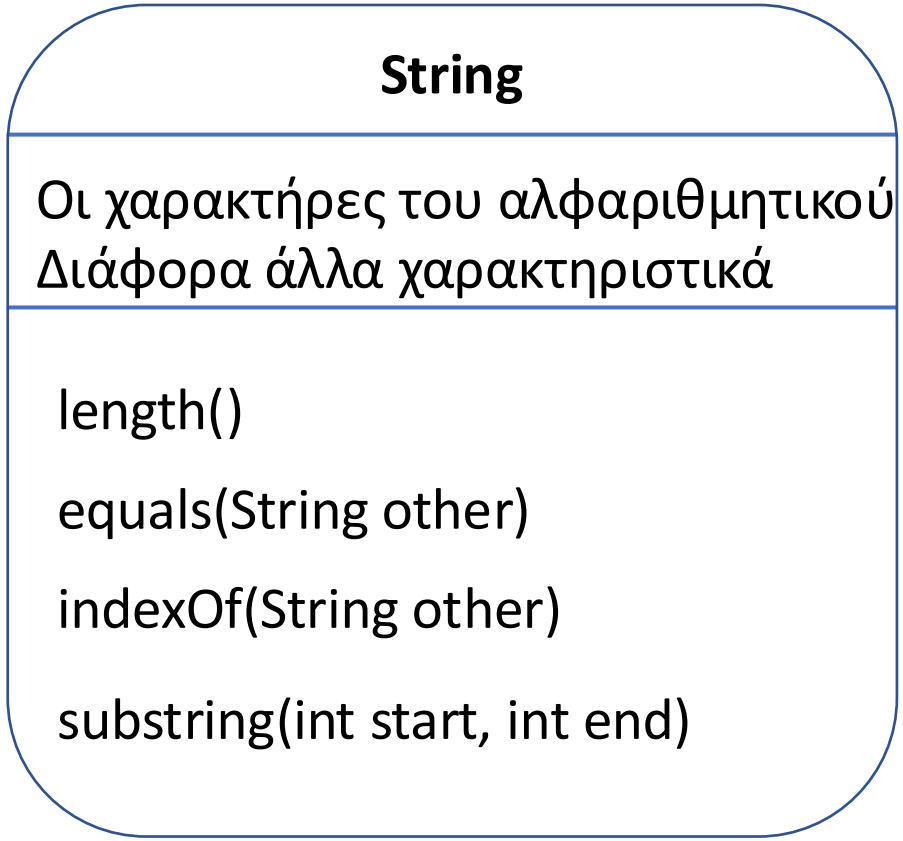
```
Light bedroomLight = new Light();
bedroomLight.on();
bedroomLight.brighten();
bedroomLight.dim();
bedroomLight.off();
```

Κλήση Μεθόδων

- Για να καλέσουμε μια μέθοδο μιας κλάσης θα πρέπει να δημιουργήσουμε ένα αντικείμενο της κλάσης
- **Εξαίρεση**: οι στατικές μέθοδοι
  - Μπορούν να κληθούν χρησιμοποιώντας το όνομα της κλάσης
- Παράδειγμα:
  - Η μέθοδος main που έχουμε δει καλείται χωρίς να έχουμε δημιουργήσει αντικείμενο
  - **Γι' αυτό και πρέπει να οριστεί ως static**

Strings

- Έχουμε ήδη χρησιμοποιήσει κλάσεις και αντικείμενα όταν χρησιμοποιούμε Strings



Η ακριβής αναπαράσταση του αλφαριθμητικού δεν έχει και τόσο σημασία εφόσον εμείς χρησιμοποιούμε μόνο τις μεθόδους

String Αντικείμενα

- Ένα String αντικείμενο είναι μια μεταβλητή τύπου String
  - Τρεις διαφορετικοί τρόποι να δώσουμε τιμή σε ένα String object

```
import java.util.Scanner;

class StringExample{
  public static void main(String[] args){
    Scanner input = new Scanner(System.in);

    String x = input.next();
    String z = new String("java");
    String y = "java";
  }
}
```

String Μέθοδοι

- Υπάρχουν πολλές χρήσιμες μέθοδοι της κλάσης String
  - length(): μήκος του String
  - equals(String x): τσεκάρει για ισότητα του String που καλεί την μέθοδο με το String x
  - trim(): αφαιρεί κενά στην αρχή και το τέλος του string
  - split(char delim): σπάει το string σε πίνακα από strings με βάσει τον χαρακτήρα delim
  - indexOf(String s): Επιστρέφει την θέση της πρώτης εμφάνισης του s μέσα στο String που καλεί την μέθοδο
  - substring(int start, int end): Επιστρέφει το υπο-string μέσα στο String που καλεί την μέθοδο μεταξύ των θέσεων start και end

Παράδειγμα

```
class StringExample{
  public static void main(String[] args){
    String x = new String("introduction to java programming");
    String y = "java";

    int offset = x.indexOf(y);
    int end = x.length();
    x = x.substring(offset,end);
    System.out.println(x);
  }
}
```

- Τα Strings είναι αμετάβλητα (immutable) αντικείμενα
- Η τελευταία ανάθεση δημιουργεί ένα καινούριο αντικείμενο και το αναθέτει στην μεταβλητή x

Αμετάβλητα Αντικείμενα

- Τα αμετάβλητα αντικείμενα (immutable objects) είναι αντικείμενα των οποίων η εσωτερική κατάσταση (ουσιαστικά τα πεδία τους) δεν μπορεί να μεταβληθεί

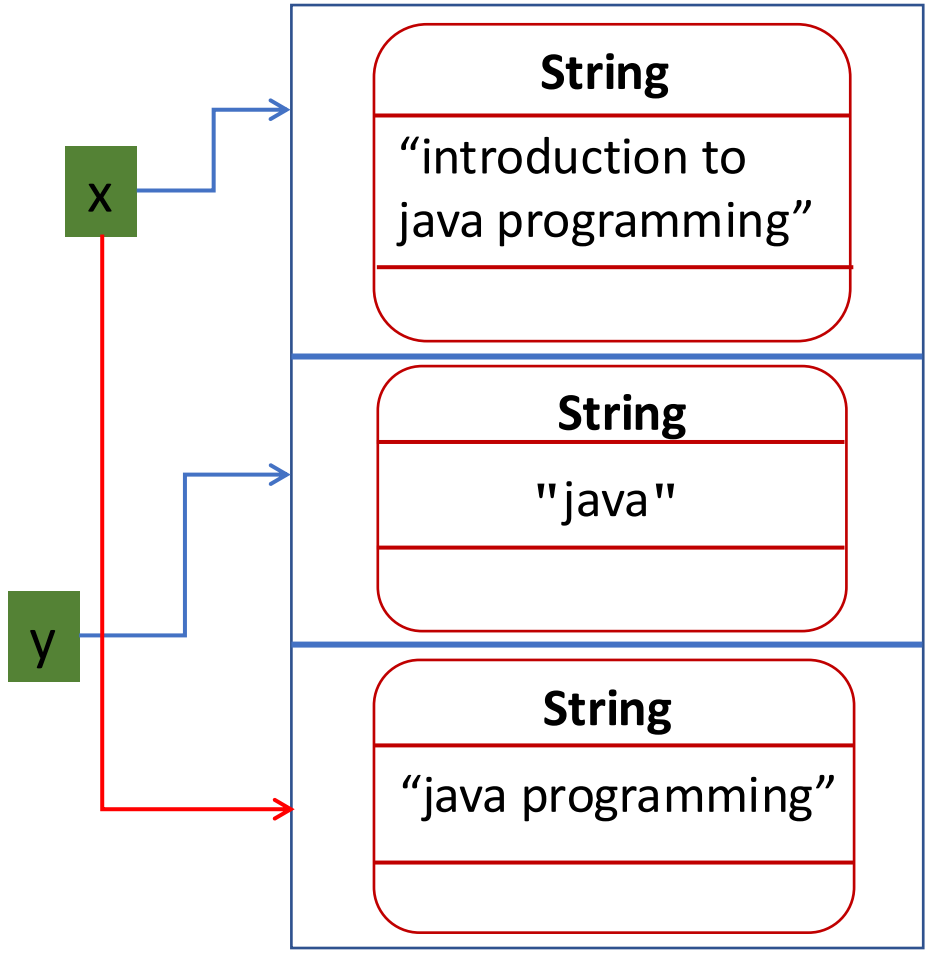
- Τα Strings είναι αμετάβλητα αντικείμενα
  - Αυτό σημαίνει ότι δεν μπορούμε να αλλάξουμε τα περιεχόμενα ενός αντικειμένου String
    - Π.χ., δεν μπορούμε να αλλάξουμε ένα χαρακτήρα ενός String
  - Ότι αλλαγή κάνουμε έχει αποτέλεσμα να δημιουργείται ένα καινούριο String και να εκχωρείται στην μεταβλητή μας

Αμετάβλητα Αντικείμενα

```
String x = new String("introduction to java programming");
String y = "java";
x = x.substring(offset,end);
```

Τα Strings είναι αμετάβλητα (immutable) αντικείμενα

Η τελευταία ανάθεση δημιουργεί ένα καινούριο αντικείμενο και το αναθέτει στην μεταβλητή x



Ισότητα String

Τι θα εκτυπωθεί?
(μια λογική συνθήκη τυπώνει true/false ανάλογα αν είναι αληθής/ψευδής)

```
import java.util.Scanner;

class StringEquality{
  public static void main(String[] args){
    String x = new String("java");
    String y = new String("java");
    String z = y;

    System.out.println("1. "+ (x == y));
    System.out.println("2. "+ (y == z));
    System.out.println("3. "+ (z == x));
    System.out.println("4. "+ x.equals(y));
    System.out.println("5. "+ y.equals(z));
    System.out.println("6. "+ z.equals(x));
  }
}
```

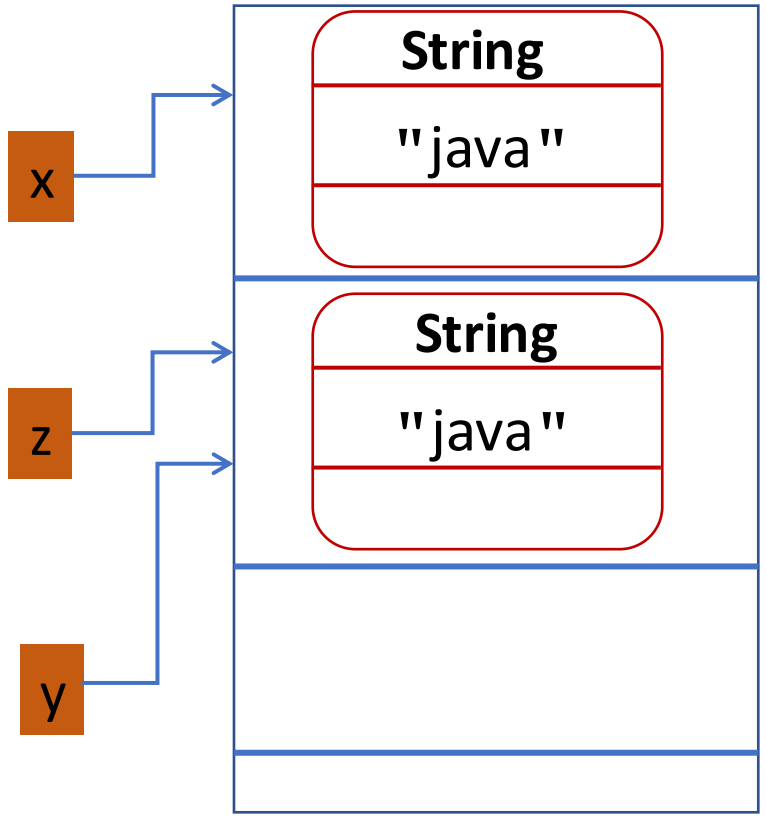
1. false
2. true
3. false
4. true
5. true
6. true

Για την σύγκριση Strings **ΠΑΝΤΑ** χρησιμοποιούμε την μέθοδο equals

Εξήγηση

- Όταν δημιουργούμε ένα String αντικείμενο δεσμεύουμε χώρο στη μνήμη για το αντικείμενο
- Η μεταβλητή που ορίζουμε «δείχνει» σε αυτό το χώρο μνήμης
- Η εκχώρηση μεταξύ αντικειμένων τα κάνει να δείχνουν στην ίδια θέση μνήμης

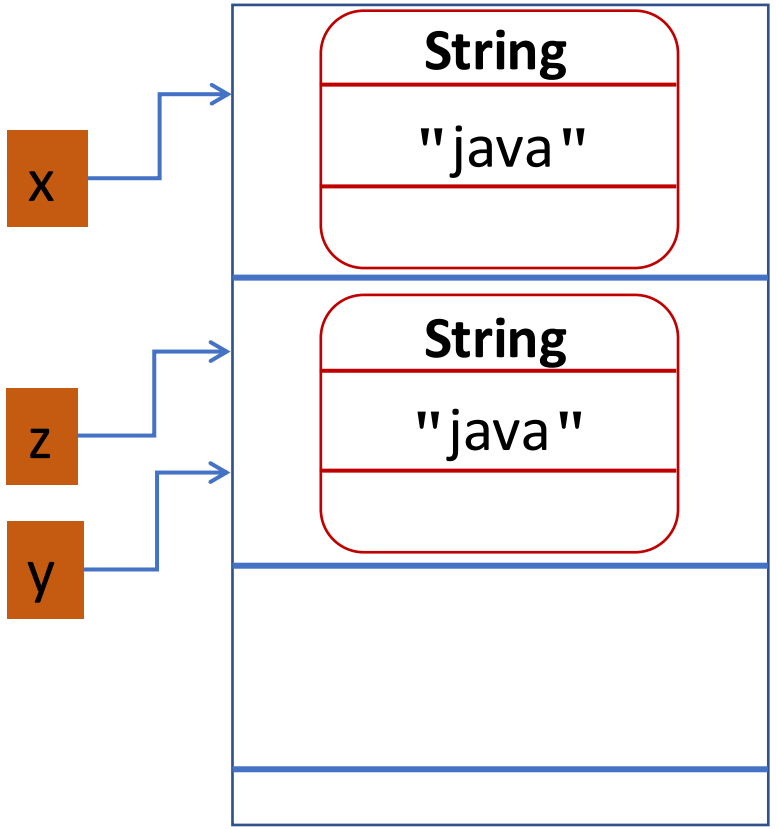
```
String x = new String("java");
String z = new String("java");
String y = z;
```



Εξήγηση

- Ο τελεστής == μεταξύ δύο αντικειμένων εξετάζει αν δείχνουν στην ίδια θέση μνήμης
- Γι' αυτό (`y == z`) επιστρέφει true

```
String x = new String("java");
String z = new String("java");
String y = z;
System.out.println("2. "+ (y == z));
```



Scanner

- Δημιουργία αντικειμένου Scanner
  - `Scanner input = new Scanner(System.in);`
- Μέθοδοι της Scanner:
  - **`next()`** : επιστρέφει το επόμενο String από την είσοδο (όλοι οι χαρακτήρες από το σημείο που σταμάτησε την προηγούμενη φορά μέχρι να βρει white space: κενο, tab, αλλαγή γραμμής)
  - **`nextInt()`** : διαβάζει το επόμενο String και το μετατρέπει σε int και επιστρέφει ένα int αριθμό
  - **`nextDouble()`** : διαβάζει το επόμενο String και το μετατρέπει σε double και επιστρέφει τον double αριθμό
  - **`nextLine()`** : Διαβάζει ότι υπάρχει μέχρι να βρει newline και το επιστρέφει ως String

Wrapper Classes

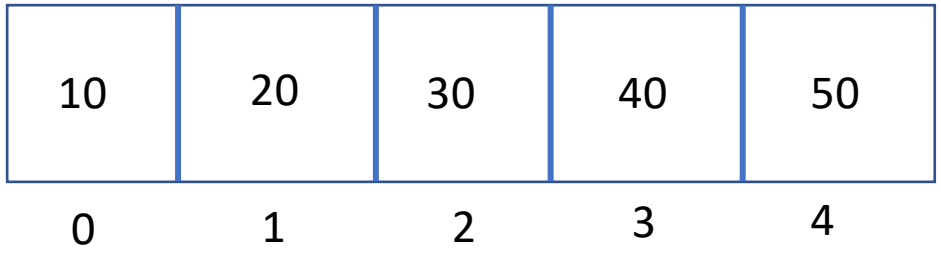
- Για κάθε βασικό τύπο η Java έχει και μία wrapper class:
  - Integer class
  - Double class
  - Boolean class
- Οι κλάσεις αυτές έχουν κάποιες μεθόδους και πεδία που μπορεί να μας είναι χρήσιμα
  - Κατά κύριο λόγο μετατροπή από και προς string
  - Τη μέγιστη και την ελάχιστη τιμή κάθε τύπου
- Κάποιες από τις μεθόδους είναι στατικές
  - Μπορούμε να τις καλέσουμε χωρίς να έχουμε αντικείμενο

Παράδειγμα

```
class WrapperTest{
  public static void main(String args[])
  {
    int i = Integer.valueOf("2");
    double d = Double.parseDouble("2.5");
    System.out.println(i*d);
    Integer x = 5;
    Double y = 2.5;
    String s = x.toString() + y.toString();
    System.out.println(s);
    System.out.println(Integer.MAX_VALUE);
  }
}
```

Πίνακες

- `int myArray[] = {10,20,30,40,50};`



- Για να προσπελαύνουμε το δεύτερο στοιχείο του πίνακα
  - `myArray[1] += 5;`
  - `System.out.println(myArray[1]);`

Πίνακες

```
public class TestArrays1 {
    public static void main(String[] args){

        int arr0[];  // int[] arr0;

        int arr1 [] = {1, 2, 3, 4};
        for (int i = 0; i < arr1.length; i++){
            System.out.println(arr1[i]);
        }

        int arr2[] = new int [10];
        for (int i = 0; i < arr2.length; i++){
            arr2[i] = i+1;
        }
        arr0 = arr2;

    }
}
```

Εναλλακτικό συντακτικό

Παράδειγμα με Strings και Πίνακες

- Φτιάξτε ένα πρόγραμμα που να διαβάζει μία γραμμή από κείμενο και να ψάχνει μία λέξη που δίνουμε σαν όρισμα μέσα σε αυτή τη γραμμή

- **`java LookFor hello`**
  - Περιμένει να διαβάσει μια γραμμή από κείμενο και ψάχνει τη λέξη hello μέσα στο κείμενο.

```
import java.util.Scanner;

class LookFor
{
    public static void main(String args[])
    {
       String name = "default";
       if (args.length == 1)
       {
             name = args[0];
       }
       Scanner input = new Scanner(System.in);
       String line = input.nextLine();
       String[] words = line.split(" ");
       for (int i=0; i < words.length; i++)
       {
           if (name.equals(words[i])){
              System.out.println(name + " found it at " + i);
           }
       }
    }
}
```

Τα command-line ορίσματα του προγράμματος αποθηκεύονται στον πίνακα από Strings που είναι όρισμα στην main()

Η μέθοδος split της κλάσης String με όρισμα ένα delimiter string σπάει το String με βάση το delimiter και επιστρέφει ένα πίνακα από Strings

Στην περίπτωση αυτή σπάμε το line με βάση το κενό και παίρνουμε τις λέξεις

Η Κλάση ArrayList

- Η κλάση ArrayList ορίζει ένας δυναμικό πίνακα με μεταβλητό μέγεθος ανάλογα με τον αριθμό των στοιχείων που τοποθετούμε
  - Το ArrayList μπορεί να κρατάει αντικείμενα οποιουδήποτε τύπου
- Συντακτικό:
  - `import java.util.ArrayList;`
  - `ArrayList< Τύπος> myList;`
- Ο τύπος είναι μια οποιαδήποτε κλάση
  - Αυτός είναι ο τύπος των δεδομένων που αποθηκεύει ο πίνακας μας
  - Για να αποθηκεύσουμε βασικούς τύπους χρειαζόμαστε την wrapper class
- Παραδείγματα:
  - `ArrayList<Integer> myList;` // λίστα από ακεραίους
  - `ArrayList<String> myList;` // λίστα από String
  - `ArrayList<Person> myList;` // λίστα από αντικείμενα Person

ArrayList

- Constructors
  - `ArrayList<T> myList = new ArrayList<T>();`
- Μέθοδοι
  - `add(T x)`: προσθέτει το στοιχειό `x` στο τέλος του πίνακα
  - `add(int i, T x)`: προσθέτει το στοιχειό `x` στη θέση `i` και μετατοπίζει τα υπόλοιπα στοιχεία κατά μια θέση
  - `get(int i)`: επιστρέφει το αντικείμενο τύπου `T` στη θέση `i`
  - `remove(int i)`: αφαιρεί το στοιχείο στη θέση `i`
  - `remove(T x)`: αφαιρεί το στοιχείο `x`
  - `set(int i, T x)`: θέτει στην θέση `i` την τιμή `x` αλλάζοντας την προηγούμενη
  - `size()`: ο αριθμός των στοιχείων του πίνακα
- Διατρέχοντας τον πίνακα:
  - `ArrayList<T> myList = new ArrayList<T>();`
  - `for(T x: myList){…}`

Παράδειγμα

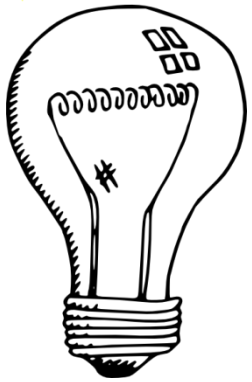
- Ζητάμε από την είσοδο ακεραίους αριθμούς μέχρι ο χρήστης να δώσει το -1. Αποθηκεύστε τους αριθμούς σε ένα πίνακα και τυπώστε τους
- Δεν ξέρουμε εκ των προτέρων πόσους αριθμούς θα πρέπει να αποθηκεύσουμε
  - Θα χρησιμοποιήσουμε ArrayList αντί για πίνακα

```
import java.util.ArrayList;
import java.util.Scanner;

class ArrayListTest
{
    public static void main(String[] args){
       ArrayList<Integer> numbers = new ArrayList<Integer>();
       Scanner input = new Scanner(System.in);
       int x = input.nextInt();
       while (x != -1){
           numbers.add(x);
           x = input.nextInt();
       }

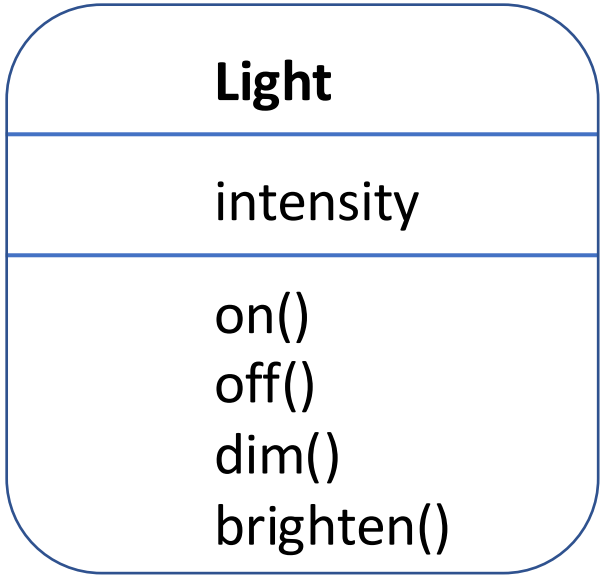
       for (Integer y: numbers){
           System.out.print(y + " ");
       }
       System.out.println();
    }
}
```

Κλάσεις Και Αντικείμενα



Θέλουμε να κάνουμε ένα πρόγραμμα που να διαχειρίζεται τα φώτα σε διάφορα δωμάτια και θα υλοποιεί και ένα dimmer





Κλάσεις Και Αντικείμενα

```
class Light
{
        private boolean lightIsOn = false;

        public void flipSwitch()
        {
                lightIsOn = !lightIsOn;
        }
}

class HouseWithLights
{
        public static void main(String[] args)
        {
                Light bedroomLight = new Light();
                bedroomLight.flipSwitch();
        }
}
```

Ορισμός κλάσης

Ορισμός (και αρχικοποίηση) πεδίου

Ορισμός μεθόδου

Χρήση πεδίου

Ορισμός αντικειμένου

Κλήση μεθόδου

Κλάσεις Και Αντικείμενα

- Ορισμός κλάσης:

```
class <Όνομα Κλάσης>
{
        <Ορισμός πεδίων κλάσης>

        <Ορισμός μεθόδων κλάσης>
}
```

- Ορισμός αντικειμένου:

```
<Όνομα Κλάσης> myObject = new <Όνομα Κλάσης>();
```

  - Ο ορισμός του αντικειμένου γίνεται συνήθως μέσα στη main ή μέσα στη μέθοδο μίας άλλης κλάσης που χρησιμοποιεί το αντικείμενο

Τα Keywords Public/Private

- Ότι είναι ορισμένο ως public σε μία κλάση είναι προσβάσιμο από μία άλλη κλάση που ορίζει ένα αντικείμενο αυτού του τύπου
  - Π.χ., η μέθοδος flipSwitch() είναι προσβάσιμη από την κλάση HouseWithLights μέσω του αντικειμένου bedroomLight
- Ότι είναι ορισμένο ως private σε μία κλάση δεν είναι προσβάσιμο από μία άλλη κλάση
  - Π.χ., το πεδίο lightIsOn δεν είναι προσβάσιμο από την κλάση HouseWithLights μέσω του αντικειμένου bedroomLight
- Μπορούμε να έχουμε public και private πεδία και μεθόδους
  - Κανόνας: Τα πεδία τα ορίζουμε (σχεδόν) ΠΑΝΤΑ private
  - Οι κλάσεις που χρειάζεται να καλούνται από αντικείμενα είναι public , αυτές που είναι βοηθητικές είναι private
- Τα πεδία και οι μέθοδοι μίας κλάσης, ανεξάρτητα αν είναι public ή private, είναι προσβάσιμα από όλες τις μεθόδους και τα αντικείμενα της ίδιας κλάσης
  - Π.χ., το πεδίο lightIsOn είναι προσβάσιμο παντού μέσα στην κλάση Light

```
class Light
{
	private boolean lightIsOn = false;

	public void flipSwitch(){
		lightIsOn = !lightIsOn;
	}

	public void printState(){
		if (lightIsOn){
			System.out.println("The light is on");
		}else {
			System.out.println("The light is off");
		}
	}
}
```

Η κατάσταση ενός αντικειμένου προσδιορίζεται από τις τιμές που έχουν τα πεδία της κλάσης

```
class HouseWithLights
{
	public static void main(String[] args){
		Light bedroomLight = new Light();
		bedroomLight.flipSwitch();
		bedroomLight.printState();
		Light kitchenLight = new Light();
		kitchenLight.flipSwitch();
		kitchenLight.printState();
	}
}
```

Η μόνη πρόσβαση που έχουμε στην κατάσταση του αντικειμένου είναι μέσω των μεθόδων της κλάσης.

Dimmer

- Θέλουμε ο διακόπτης μας να μας δίνει την δυνατότητα να αυξομειώνουμε την ένταση
  - Τι επιπλέον πεδία πρέπει να προσθέσουμε?
  - Τι επιπλέον μεθόδους χρειαζόμαστε?

```
class DimmerLight
{
    private boolean lightIsOn = false;
    private int intensity = 100;

    public void flipSwitch(){
        lightIsOn = !lightIsOn;
    }

    public void dim(){
        if (intensity > 0){
            intensity --;
        }
    }

    public void brighten(){
        if (intensity < 100){
            intensity ++;
        }
    }

    public void printState(){
        if (lightIsOn){
            System.out.println("The light is ON with intensity " + intensity);
        }else{
            System.out.println("The light is OFF");
        }
    }
}
```

```
class HouseWithDimmerLights
{
    public static void main(String[] args){
        DimmerLight bedroomLight = new DimmerLight();
        bedroomLight.flipSwitch();
        bedroomLight.dim();
        bedroomLight.printState();
    }
}
```

Dimmer

- Κάθε φορά που αυξάνουμε ή μειώνουμε την ένταση θέλουμε να μας λέει και την κατανάλωση
  - (Κατανάλωση = ένταση * 0.1 λεπτά/ώρα)

```
class DimmerLight2
{
    private boolean lightIsOn = false;
    private int intensity = 100;

    public void flipSwitch(){
          lightIsOn = !lightIsOn;
    }
    public void dim(){
          if (intensity > 0){
              intensity --;
          }
          double consumption = intensity *0.1;
          System.out.print("Consumption = "+consumption);
    }
    public void birghten(){
          if (intensity < 100){
              intensity ++;
          }
          double consumption = intensity *0.1;
          System.out.print("Consumption = "+consumption);
    }
    public void printState(){
          if (lightIsOn){
              System.out.println("The light is ON with intensity " + intensity);
          }else{
              System.out.println("The light is OFF");
          }
    }
}
```

Οι μεταβλητές consumption είναι τοπικές μεταβλητές

Υπάρχουν μόνο μέσα στις μεθόδους dim και brighten και όταν τελειώσει η κλήση τους εξαφανίζονται

Τοπικές Μεταβλητές

- Είδαμε πρώτη φορά τις τοπικές μεταβλητές όταν μιλήσαμε για μεταβλητές που ορίζονται μέσα σε ένα λογικό block
  - Παρόμοια είναι και για τις μεταβλητές μιας μεθόδου
- Τοπικές μεταβλητές μιας μεθόδου είναι οι μεταβλητές που ορίζονται μέσα στον κώδικα της μεθόδου
  - Περιλαμβάνουν και τις μεταβλητές που κρατάνε τις παραμέτρους της μεθόδου
- Οι μεταβλητές αυτές έχουν εμβέλεια μόνο μέσα στην μέθοδο
  - Εξαφανίζονται όταν βγούμε από τη μέθοδο
- Αντιθέτως τα πεδία της κλάσης διατηρούνται όσο υπάρχει το αντικείμενο, και έχουν εμβέλεια σε όλη την κλάση

Παράδειγμα 1

- Θέλουμε ένα πρόγραμμα που να προσομοιώνει την κίνηση ενός αυτοκινήτου, το οποίο κινείται πάνω σε μία ευθεία πάντα κατά μία θέση, και τυπώνει τη θέση του

MovingCar

```
class Car
{
    private int position = 0;

    public void move(){
        position += 1;
    }

    public void printPosition(){
        System.out.println("Car at position "+ position);
    }
}

class MovingCar
{
    public static void main(String args[]){
        Car myCar = new Car();
        myCar.move();
        myCar.printPosition();
    }
}
```

Ορισμός κλάσης

Ορισμός (και αρχικοποίηση) πεδίου

Ορισμός μεθόδου

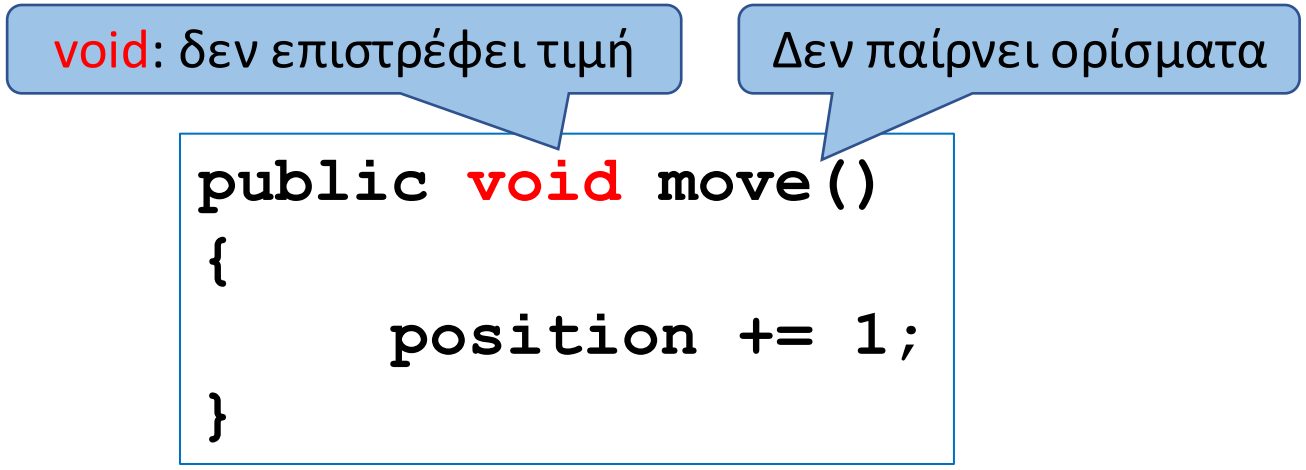
Χρήση πεδίου

Ορισμός αντικειμένου

Κλήση μεθόδου

Μέθοδοι

- Οι μέθοδοι που έχουμε δει μέχρι τώρα είναι πολύ απλές
  - Δεν έχουν παραμέτρους (δεν παίρνουν ορίσματα)
  - Δεν επιστρέφουν τιμή



Παράδειγμα 2

- Θέλουμε να μπορούμε να μετακινήσουμε το όχημα όσες θέσεις θέλουμε είτε προς τα δεξιά (+) είτε προς τα αριστερά (-)

- Για να το κάνουμε αυτό η move θα πρέπει να παίρνει σαν παράμετρο τον αριθμό των θέσεων

Παράμετροι

- Οι μέθοδοι μπορούν να έχουν παραμέτρους
  - Μας επιτρέπουν να περάσουμε τιμές στην μέθοδο μας

```
public void moveManySteps(int steps)
{
        position += steps;
}
```

Ορισμός παραμέτρου

- Μία παράμετρος ορίζεται όπως οποιαδήποτε άλλη μεταβλητή
  - Πρέπει να έχει συγκεκριμένο τύπο και όνομα
  - Είναι τοπική μεταβλητή της μεθόδου

```
int x = 10;
myCar.moveManySteps(x);
myCar.moveManySteps(10);
```

Όρισμα στην κλήση της μεθόδου

- Όταν καλούμε την μέθοδο, περνάμε ένα όρισμα
  - Θα πρέπει να συμφωνεί στον τύπο με την παράμετρο
  - Είναι σαν να κάνουμε ανάθεση steps = x ή steps = 10

```
class Car
{
    private int position = 0;

    public void moveManySteps(int steps)
    {
        position += steps;
    }
}

class MovingCar2
{
    public static void main(String args[])
    {
        Car myCar = new Car();
        int x = 10;
        myCar.moveManySteps(x);
        myCar.moveManySteps(10);
        myCar.moveManySteps(2*x+10);
    }
}
```

Στον ορισμό της μεθόδου ορίζουμε και την παράμετρο της μεθόδου, όπως ορίζουμε μια μεταβλητή. Έχει ένα τύπο και ένα όνομα

Όταν καλούμε την μέθοδο περνάμε μια τιμή σαν όρισμα στην μέθοδο
Σαν όρισμα μπορεί να είναι μια οποιαδήποτε έκφραση.
Αρκεί ή αποτίμηση της έκφρασης να έχει τύπο συμβατό με αυτόν της παραμέτρου (int στην περίπτωση μας)

Πέρασμα Παραμέτρων

- Όταν καλούμε μια μέθοδο με μία τιμή σαν όρισμα, ουσιαστικά εκχωρούμε αυτή την τιμή στην παράμετρο της μεθόδου

Η κλήση όπου η μεταβλητή x έχει την τιμή 10

```
myCar.moveManySteps(2*x+10);
```

Ισοδυναμεί με τον κώδικα:

```
{
        int steps = 30;
        position += delta;
}
```

Αποτιμάται η τιμή της έκφρασης και εκχωρείται

Το πέρασμα μεταβλητών με αυτό τον τρόπο λέγεται πέρασμα δια τιμής (pass by value). Η μέθοδος δεν έχει πρόσβαση στην μεταβλητή μόνο στην τιμή

Πέρασμα παραμέτρων δια τιμής (call by value)

- Όταν το πέρασμα παραμέτρων γίνεται δια τιμής, το πρόγραμμα μας έχει πρόσβαση μόνο στην τιμή της παραμέτρου και όχι στην μεταβλητή που χρησιμοποιήσαμε στο όρισμα
  - Σε όλες τις γλώσσες πλέον το πέρασμα παραμέτρων γίνεται δια τιμής
- Αν η παράμετρος είναι ένα αντικείμενο τα πράγματα γίνονται πιο σύνθετα
  - Η τιμή της μεταβλητής που έχουμε σαν παράμετρο είναι διεύθυνση μνήμης. Δεν μπορούμε να αλλάξουμε την διεύθυνση μνήμης αλλά μπορούμε να αλλάξουμε τα περιεχόμενα της

```
class Car
{
    private int position = 0;

    public void moveManySteps(int steps, String direction)
    {
       if (direction.equals("right"){ position += steps;}
       if (direction.equals("left") { position -= steps;}
    }
}

class MovingCar3
{
    public static void main(String args[]){
       Car myCar = new Car();
       myCar.moveManySteps(10, "left");
    }
}
```

Μέθοδος με πολλές παραμέτρους

Τα ορίσματα θα πρέπει να συμφωνούν με το πλήθος και τους τύπους των παραμέτρων στην αντίστοιχη θέση

Κλήση της μεθόδου

Τύποι παραμέτρων και ορισμάτων

- Οι παράμετροι μιας μεθόδου έχουν συγκεκριμένο τύπο
- Τα ορίσματα στην κλήση της μεθόδου θα πρέπει να συμφωνούν με τον τύπο της παραμέτρου, θέση προς θέση
- Ισχύουν οι μετατροπές τύπου που ξέρουμε
  - byte → short → int → long → float → double
- Μία μέθοδος μπορεί να πάρει ως όρισμα και ένα αντικείμενο μιας κλάσης
  - Το πώς δουλεύει αυτό θα το μάθουμε όταν μιλήσουμε για αναφορές

Μέθοδοι που επιστρέφουν τιμές

- Μέχρι τώρα οι μέθοδοι που φτιάξαμε δεν επιστρέφουν τιμή
  - Είναι τύπου void
- Σε πολλές περιπτώσεις θέλουμε η μέθοδος να μας επιστρέφει τιμή
  - Π.χ., μία μέθοδος που υπολογίζει το άθροισμα δύο αριθμών

Παράδειγμα 3

- Το αυτοκίνητο μας δεν μπορεί να μετακινηθεί έξω από το διάστημα [-10,10]. Θέλουμε η moveManySteps να μας επιστρέφει μια λογική τιμή αν η μετακίνηση έγινε η όχι

```
class Car
{
    private int position = 0;

    public boolean moveManySteps(int steps)
    {
       if ((position + steps < -10) || (position + steps > 10)){
          return false;
       }else{
           position += steps;
           return true;
       }
    }
}
```

Όταν ορίζουμε μια μέθοδο που επιστρέφει τιμή θα πρέπει να ορίσουμε τον τύπο της τιμής που επιστρέφει.

Π.χ. αυτή η μέθοδος επιστρέφει τιμή boolean

Μια μέθοδος μπορεί να επιστρέφει και ένα αντικείμενο μιας κλάσης

Επιστρέφουμε μια τιμή μέσα στον κώδικα χρησιμοποιώντας την εντολή return

Η Εντολή return

- Η εντολή return χρησιμοποιείται για να επιστρέψει μια τιμή μια μέθοδος
- Συντακτικό:
  - `return <έκφραση>`
- Κάθε μονοπάτι εκτέλεσης του κώδικα θα πρέπει να επιστρέφει μια τιμή
- Η κλήση της return σε οποιοδήποτε σημείο του κώδικα σταματάει την εκτέλεση της μεθόδου και επιστρέφει τιμή
  - Μπορούμε να το χρησιμοποιήσουμε αυτό για να απλοποιήσουμε τον κώδικα

```
class Car
{
    private int position = 0;

    public boolean moveManySteps(int steps)
    {
       if ((position + steps < -10) || (position + steps > 10)){
            return false;
       }
       position += steps;
       return true;
    }
}
```

Γιατί δεν έχουμε else εδώ?

Απλά δεν χρειάζεται

Αν μπούμε μέσα στο if η return θα σταματήσει την εκτέλεση του κώδικα και θα μας βγάλει από την μέθοδο. Επιστρέφεται η τιμή false.

Ο Τύπος μιας Μεθόδου

- Μια μέθοδος που επιστρέφει τιμή ορίζεται με συγκεκριμένο τύπο. Π.χ.
  - `public boolean moveManySteps(int steps)`
  - `public double division(int x, int y)`
  - `public String getUserName()`
  - `public Car getCar()`
- Αν έχουμε μια συνάρτηση που επιστρέφει τιμή τύπου T
  - Π.χ. `public double division(int x, int y)`

    Η έκφραση στο return πρέπει να επιστρέφει μία τιμή τύπου (συμβατού με το) T (π.χ., `return x/(double)y`)

```
import java.util.Scanner;

class Car
{
    private int position = 0;
    public boolean moveManySteps(int steps){
        if ((position + steps < -10) || (position + steps > 10)){
            return false;
        }
        position += steps;
        return true;
    }
    public void printPosition(){
        System.out.println("Car at position "+position);
    }
}

class MovingCar4b{
    public static void main(String args[]){
        Scanner input = new Scanner(System.in);
        Car myCar = new Car();
        int steps = input.nextInt();
        boolean carMoved = myCar.moveManySteps(steps);
        if (carMoved) { myCar.printPosition();}
        else { System.out.println("Car could not move");}
    }
}
```

Κλήση της μεθόδου

```
import java.util.Scanner;

class Car
{
    private int position = 0;

    public boolean moveManySteps(int steps)
    {
        if ((position + steps < -10) || (position + steps > 10)){
            return false;
        }
        position += steps;
        return true;
    }
    public void printPosition(){
        System.out.println("Car at position "+position);
    }
}

class MovingCar4c
{
    public static void main(String args[]){
        Scanner input = new Scanner(System.in);
        Car myCar = new Car();
        int steps = input.nextInt();
        myCar.moveManySteps(steps);
        myCar.printPosition();
    }
}
```

Δεν είναι υποχρεωτικό να χρησιμοποιούμε πάντα την επιστρεφόμενη τιμή

Η moveManySteps επιστρέφει τιμή, αλλά η κλήση της την αγνοεί

Η printPosition θα επιστρέψει 0 αν δεν κινήθηκε το όχημα

Η Εντολή return

- Μπορούμε να καλέσουμε την return και σε μία void μέθοδο
  - Χωρίς επιστρεφόμενη τιμή
    - `return;`
  - Σταματάει την εκτέλεση της μεθόδου

```
public void printIfPositive()
{
    if (position < 0){
        return;
    }
    System.out.println("position = " + position);
}
```

Η Εντολή return

- Μπορούμε να καλέσουμε την return και σε μία void μέθοδο
  - Χωρίς επιστρεφόμενη τιμή
    - `return;`
  - Σταματάει την εκτέλεση της μεθόδου

```
public void moveManySteps(int steps, String direction)
{
        if (steps < 0){
                return;
        }
        if (direction.equals("right"){ position += steps;}
        if (direction.equals("left") { position -= steps;}
}
```

Παράδειγμα 4

- Θέλουμε να μπορούμε να κινούμε το όχημα όσες θέσεις θέλουμε είτε προς τα δεξιά (+) είτε προς τα αριστερά (-), και να τυπώνεται η θέση σε κάθε κίνηση

- Υλοποίηση: Θα ορίσουμε μια βοηθητική μεταβλητή delta την οποία θα προσθέτουμε στο position σε κάθε βήμα. Η default τιμή του θα είναι delta = 1. Αν η παράμετρος steps είναι αρνητική θα την μετατρέπουμε σε θετική και θα θέσουμε delta = -1

```
class Car
{
   private int position = 0;

   public void moveManySteps(int steps)
   {
      int delta = 1;
      if (steps < 0){
        steps = -steps; delta = -1;
      }
      for (int i = 0; i < steps; i ++){
        position += delta;
        System.out.println("Car at position "+position);
      }
   }
   public void printPosition(){
      System.out.println("Car at position "+position);
   }
}

class MovingCar5
{
   public static void main(String args[]){
      Car myCar = new Car();
      int steps = -10;
      myCar.moveManySteps(steps);
      System.out.println("--: " + steps);
   }
}
```

Το **delta** είναι τοπική μεταβλητή της μεθόδου. Ορίζεται μέσα στην μέθοδο και υπάρχει μόνο μέσα στην μέθοδο. Στο τέλος της μεθόδου η μεταβλητή χάνεται.

Η παράμετρος **steps** λειτουργεί ως τοπική μεταβλητή της συνάρτησης και χάνεται μετά την κλήση της μεθόδου.

Η μεταβλητή **steps** στην main είναι διαφορετική από την παράμετρος **steps** .
Το πέρασμα παραμέτρων γίνεται δια τιμής και άρα η τιμή της μεταβλητής του ορίσματος δεν μεταβάλλεται

Τυπώνει **--:-10**

```
class Car
{
    private int position = 0;

    public void moveManySteps(int steps)
    {
       int delta = 1;
       if (steps < 0){
           steps = -steps; delta = -1;
       }
       for (int i = 0; i < steps; i ++){
           position += delta;
           printPosition();
       }
    }

    public void printPosition(){
       System.out.println("Car at position "+position);
    }
}
```

Μπορούμε να κάνουμε την εκτύπωση καλώντας την printPosition()

Κάθε μέθοδος που ορίζουμε μέσα σε μία κλάση μπορούμε να την χρησιμοποιήσουμε και μέσα στην κλάση